Nested Classes and Subclasses

Where a class lives compared with what a class is

This addendum started with a student's question. After we began inheritance, a student asked how a nested class is different from a subclass. It is a fantastic question. Both ideas involve one class being connected to another, both came up this semester within a few weeks of each other, and the words "inner class" and "subclass" sound like they should mean almost the same thing. They do not, and working out exactly why is a good way to understand both. Questions like this one, where you notice that two ideas look alike and stop to ask how they really differ, are how understanding gets deeper. Please keep asking them.

This addendum goes with Part 1 of the inheritance notes and with the first set of review notes, Static Members and Nested Classes. It uses examples from both.

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. The Short Answer

The two ideas answer different questions.

- **A nested class** is about *where* a class is declared. It is written inside another class's body. That affects its name, who can see it, and, for an inner class, which outer object it belongs to.
- **A subclass** is about *what type* a class is. It extends another class, so it inherits that class's members and can be used anywhere that class is expected.

The two ideas are independent of each other. A class can be nested, a subclass, both, or neither.

2. Nested Classes, a Quick Review

In the review notes, `Roster` held two nested classes:

```
public class Roster {
    private final String courseCode;
    private final ArrayList<Student> students = new ArrayList<>();

    public static class Stats { ... }   // static nested class
    private class Report { ... }        // inner class
```

```
}
```

- Its full name includes the outer class: `Roster.Stats`.
- It can be `private`, which hides it from every class outside `Roster`.
- It can read the outer class's `private` members, because it is part of the same class body. `Report` reads `courseCode` and `students` directly.
- An inner class (one without `static`) is linked to one particular `Roster` object through a hidden reference. That relationship is closer to has-a than is-a: a `Report` *belongs to* a roster. It is not *a kind of* roster.
- It inherits nothing from `Roster`. A `Report` is not a `Roster`, and you cannot store one in a `Roster` variable.

3. Subclasses, a Quick Review

In Part 1 of the inheritance notes, `Manager` extended `Employee`:

```
public class Manager extends Employee { ... }
```

- It is a separate class, usually in its own file, with its own name.
- It inherits `Employee`'s fields (as data) and its accessible methods, and it can override those methods.
- It *cannot* read `Employee`'s `private` members. It has to use `Employee`'s methods, or call them with `super`.
- A `Manager` object is one object that contains the `Employee` part. There is no separate outer object and no hidden link.
- It *is an* `Employee`, so polymorphism applies: a `Manager` can go in an `Employee` variable, array, list, or parameter.

4. Seeing the Difference in Code

Here is a small `Employee` class with a static nested class, `Badge`, plus a `Manager` subclass. The two relationships behave in opposite ways.

```
public class Employee {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    // A static nested class: declared inside Employee, but not a kind of Employee.
    public static class Badge {
        public static String print(Employee e) {
            return "STAFF: " + e.name;   // compiles: same class body
        }
```

```
    }
}

public class Manager extends Employee {
    public Manager(String name) {
        super(name);
    }
}
```

```
Employee ada = new Employee("Ada");
Employee grace = new Manager("Grace");     // a Manager is an Employee
System.out.println(Employee.Badge.print(ada));
System.out.println(Employee.Badge.print(grace));
```

```
STAFF: Ada
STAFF: Grace
```

Now try two things the relationships do not allow. First, the subclass reaching for the `private` field:

```
public String label() {   // inside Manager
    return name;          // does not compile
}
```

```
error: name has private access in Employee
```

Second, treating the nested class as if it were a kind of `Employee`:

```
Employee e = new Employee.Badge();   // does not compile
```

```
error: incompatible types: Badge cannot be converted to Employee
```

So the nested class has the access, but not the type. The subclass has the type, but not the access.

5. Side by Side

Table 1: Nested classes compared with subclasses.

Question	Nested class	Subclass
What it describes	Where the class is declared	What type the class is
How you write it	Inside another class's body, with or without `static`	With `extends`
Relationship	Belongs to the outer class; an inner class belongs to one outer object	Is-a
Inherits the other class's members?	No	Yes
Reads the other class's `private` members?	Yes	No
Usable where the other class is expected?	No	Yes
How many objects?	An inner object and its outer object are two separate objects	One object, which contains the superclass part

6. When a Class Is Both

Because the ideas are independent, one class can be nested *and* a subclass. You have already seen two examples.

- `ConstructorOrderDemo` **in Part 3.** `Base` and `Derived` are both static nested classes inside `ConstructorOrderDemo`, and `Derived extends Base`. The nesting only keeps the whole demo in one file. The `extends` is what makes the constructors chain.
- **The anonymous class in the review notes.** The expression `new StudentFilter() { ... }` declares a class inside a method, so it is nested. That same class implements `StudentFilter`, so it can be passed anywhere a `StudentFilter` is expected. An anonymous class can extend a class in the same way.

In each case, ask the two questions separately: where is it declared, and what type is it?

7. A Quick Test

When you are deciding which one you need, ask:

1. **Is this thing a kind of the other class?** If yes, and it should be usable anywhere the other class is, you want a subclass.
2. **Does it exist only to help the other class?** If yes, a nested class keeps it close and lets you hide it. Start with a static nested class.
3. **Does it belong to one particular object of the other class?** If yes, an inner class gives it that link to the outer object.

8. Try It

1. Move `Badge` out of `Employee` and into its own file, as an ordinary top-level class. Which line stops compiling, and why? What does that tell you about what nesting gave `Badge`?
2. Write a `private static class` inside `Manager` that formats a manager's title. Is that class a subclass of anything? What can it see?
3. For each of these, decide whether a subclass, a static nested class, or an inner class fits best, and explain why in one sentence: a `Truck` in a rental program; the `Stats` summary of a `Roster`; an object that steps through the items of one particular list.